

Data Structures and Algorithms

Module 1

Let me know, if any typos or any other type of mistakes are in shared PPT.

Module No. 1

Notion of an Algorithm, Fundamentals of Algorithmic Problem Solving, Important Problem Types, Fundamentals of the Analysis of Algorithm Efficiency, Analysis Framework, Asymptotic Notations and its properties, Mathematical analysis for Recursive and Non-recursive algorithms.

- An algorithm is a step by step procedure for solving the given problem.
- An algorithm is independent of any programming language and machine.
- An Algorithm is a finite sequence of effective steps to solve a particular problem where each step is unambiguous(definite) and which terminates for all possible inputs in a finite amount of time.
- An Algorithm accepts zero or more inputs and produces one or more outputs.

The criteria for any set of instruction for an algorithm is as follows:

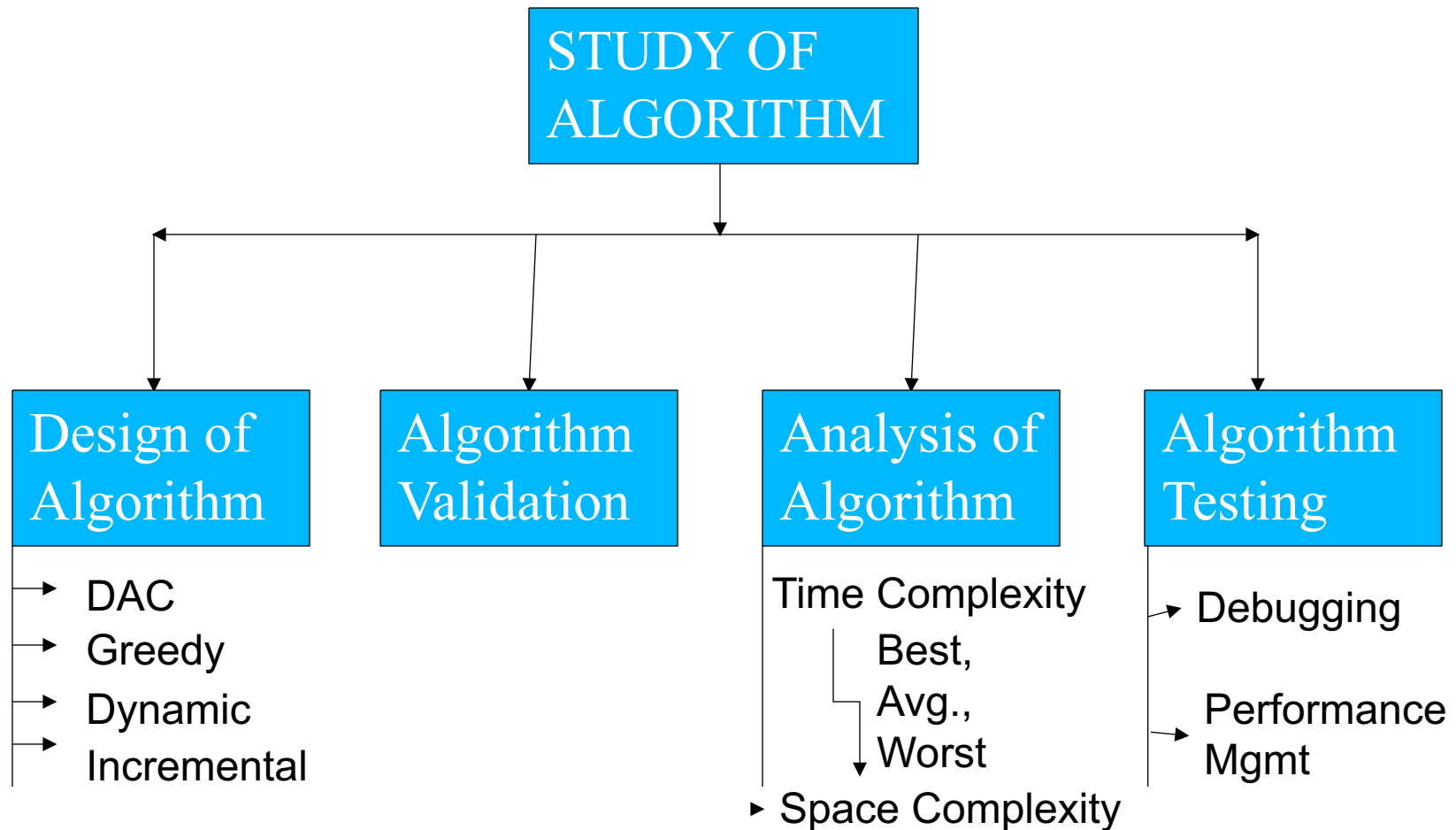
- 1. Input** : Zero or more quantities that are externally applied
- 2. Output** : At least one quantity is produced
- 3. Definiteness** : Each instruction should be clear and unambiguous
- 4. Finiteness** : Algorithm terminates after finite number of steps for all test cases.
- 5. Effectiveness** : Each instruction is basic enough for a person to carried out using a pen and paper. That means ensure not only definite but also check whether feasible or not.

- Each instruction should be clear and unambiguous.
- It must be perfectly clear what should be done.
- Example Directions which are not permitted
 - “add 6 or 7 to x”
 - compute $x/0$
 - remainder of m/n when m and n are –ve numbers
- it is not clear which of the 2 possibilities should be done.
- Achievement using programming language for algorithms designed such that each legitimate sentence has a unique meaning

Effectiveness

- Each step must be such that it can at least in principle be done by a person using paper and pencil in a finite amount of time.
- Example -Effective
- Performing arithmetic on integers
- Example not effective
- Some arithmetic with real numbers.

- The algorithm should terminate after a finite number of steps in all the cases.
- The time for termination should be reasonably short.



- **Design:** Various design techniques are available which yield good algorithm. **Divide and conquer**, incremental approach, **greedy**, **dynamic programming** etc.
- **Divide and Conquer:** Most of the algorithms are recursive in nature. That means, for solving a particular problem they call themselves repeatedly one or more times. All these algorithm follows DAC approach.
- In this approach, whole problem is divided into several sub problems. These sub problems are similar to the original problem but smaller in size. All these sub problems are solved recursively. The solution obtained from these sub problems are then combined to create a solution to the original problem.
- **Step 1: Divide-** In this step whole problem is divided into several sub problems.
- **Step 2: Conquer-** The sub problems are conquered by solving them recursively only if they are small enough to be solved.
- **Step 3: Combine-** Finally the solution obtained by the sub problems are combined to create solution to the original problems.

- **Analysis:** Analysis of algorithm depends upon various factors such as memory, communication bandwidth, or computer hardware.
- But the most often used is the **Computation time** that an algorithm requires for completing the given task.
- The analysis of algo. Focuses on **time and space complexity**.
- For an algo. the **time complexity** depends upon the size of the input, thus it is a function of i/p size “n”. The amount of time needed by an algo. to run to completion.
- It may be noted that different time can arise for the same algorithm. Usually we deal with Best, Avg, and Worst case

- **Best Case:** The minimum amount of time that an algorithm requires for an input size n is referred to as Best case time complexity.
- **Avg. Case:** It is the execution of an algorithm having typically input data of size n .
- **Worst Case:** The maximum amount of time needed by an algorithm for an input size n is referred to as worst case time complexity.
- The amount of **memory** needed by program to run to completion is referred to as **space complexity**.

Good algorithm

- One which works correctly
 - should give the desired output
- Readable
 - Steps are clear and understandable
- Efficient in terms of time and memory utilization .
 - give the desired output faster
 - utilize less resources.
- Given multiple solutions, finding out the better one.

Mathematical Formulae Contd..

$$\log_a xy = \log_a x + \log_a y$$

$$\log_a (x/y) = \log_a x - \log_a y$$

$$\log_a x^n = n \log_a x$$

$$\log_a 1 = 0$$

$$\log_x x = 1$$

$$\log_a x = 1 / \log_x a$$

$$\log_a(x) = \log_a(b) * \log_b(x)$$

$$\log_a x = \log_b x / \log_b a$$

$$a^{\log_a x} = x$$

$$\text{if } 2^k = n \text{ then } k = \log_2 n$$

$$x^{\log_b y} = y^{\log_b x}$$

Space Time Tradeoff Example

Store Employee information

- Solutions
 - Array
 - Linked list
- Which locates a particular employee faster?
- Which provides better utilization of memory?
- Which is easy to code?
- Which is easier to test?

Space Complexity

- Amount of memory the algorithm needs to run
Requirements
 - Fixed part
 - Variable part

Example Algorithm

```
Algorithm sum(a,n)
{ s = 0.0;
for i:=1 to n do s:=s+a[i];
return s; }
```

one word for each integer $n, i, s, a[]$

$s(n) \geq n+3$

Example Algorithm



```
algorithm Rsum(a,n)
{ if (n<=0) then
    return 0.0
  else
    return Rsum(a,n-1)+a[n];
}
```

- Instance characterized by n
- Recursive stack space required at each level
 - $(n, \text{return address}, *a[])$ 3 words
- Depth of recursion considering first invocation - $(n+1)$
- $S(n) \geq 3 * (n+1)$

Time Complexity

- Amount of computer time needed for the algorithm to run to completion
- $T(P) = C(P) + R(P)$
- $C(P)$ is independent of problem instance and is always not required (recompilation)
- So usually only $R(P)$ is considered

Apriori Measures

- Instruction count
 - Basic operations
 - Basic steps
- Size of input.

Computing Instruction Count

- Initialization instructions
- Loops.
 - Number of passes of the loops
- Count the basic operations/steps

Counting Steps

- Time complexity can also be expressed in terms of the basic steps performed.
- Expressed as a function of instance characteristics
 - input size
 - output size etc .

Counting Steps Implementation

- Assignment – count 1
- For, while, repeat loops
 - Using a variable count
 - Using steps per execution of instructions

Use of a variable count

- Use a variable count to store step count and increment it wherever necessary
- Display count

Computing Step Count from s/e

- Each statement will have an (s/e) steps/execution depending on the type of statement
- Frequency of each statement is multiplied with the corresponding s/e of that statement to get the step count for that statement.
- Statement Step counts are added to give the total count

$$\sum f \times (s/e)$$

Example 1

Algorithm sum(a,n)

```
{ s=0.0;  
for i: =1 to n do  
{ s:=s+a[i]; }  
return s; }
```

s/e	freq	total
1	1	1
1	n+1	n+1
1	n	n
1	1	1

		2n+3

Example 2

```
algorithm add(a,b,b,m,n)
```

		S/e	Freq	Total
{				
for i:=1 to m do	→	1	m+1	m+1
for j:=1 to n do	→	1	m(n+1)	m(n+1)
c[i,j]:=a[i,j]+b[i,j];	→	1	mn	mn
}				-----
				2mn+2m+1



End



Design and Analysis of Algorithm

Module 1.1

Dr. K Hemant Reddy
SCOPE

VIT AP UNIVERSITY, Andhra Pradesh

26

Example 2



algorithm add(a,b,b,m,n)	S/e	freq	= Total
{for i:=1 to m do	1	m+1	= m+1
for j:=1 to n do	1	m(n+1)	= m(n+1)
c[i,j]:=a[i,j]+b[i,j];	1	mn	= mn
}			-----
			2mn+2m+1

VIT AP UNIVERSITY, Andhra Pradesh

s/e freq = total

$$1 \quad 1 = 1$$
$$1 \quad n+1 \quad = \quad n+1$$
$$1 \cdot n * n = n^2 + n$$

1 $n * n * n = n^3 + n^2$

$$1 \cdot n \cdot n = n^3$$

.....

$$= 2n^3 + 2n^2 + 2n + 2$$

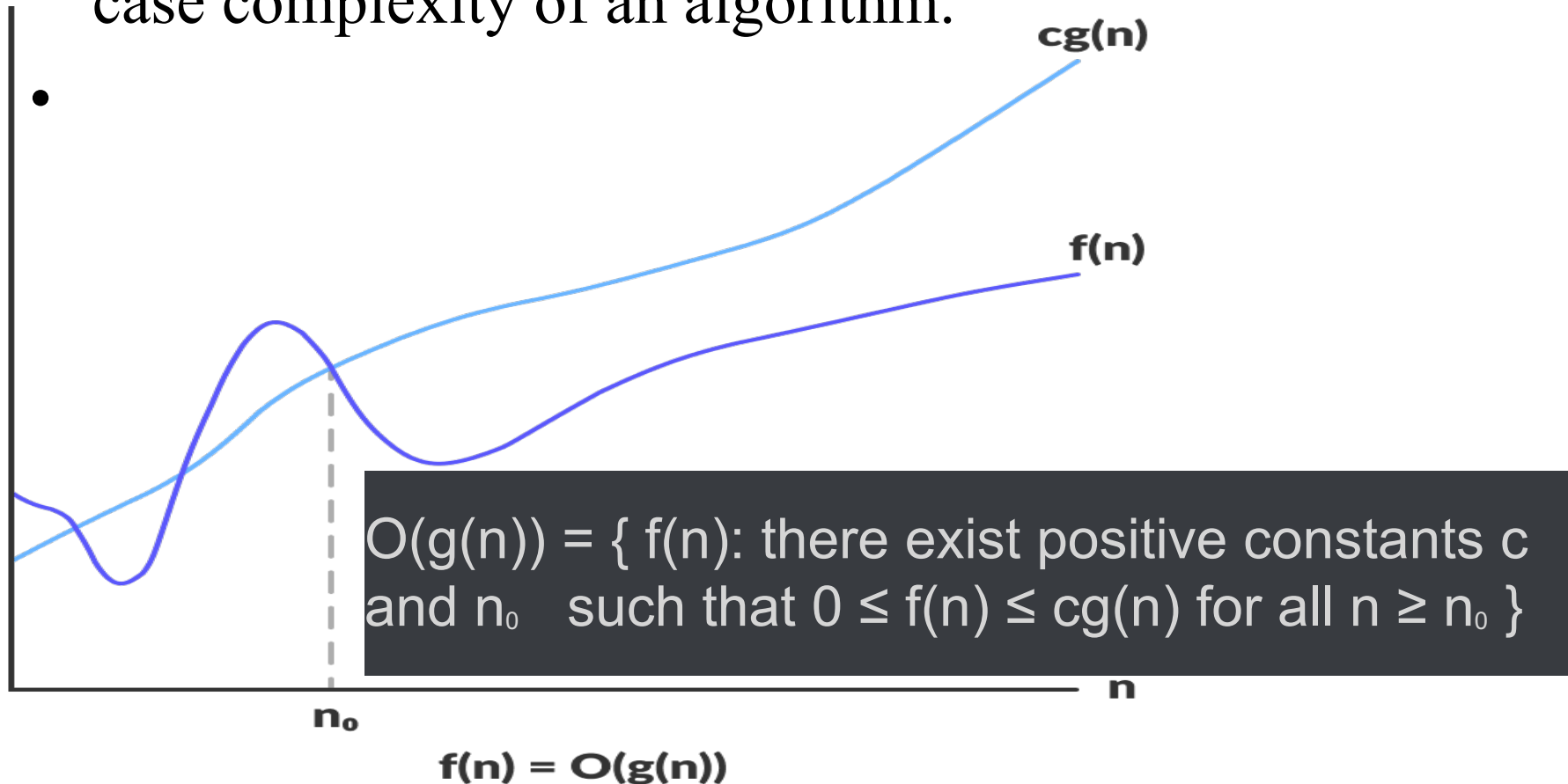
- The efficiency of an algorithm depends on the amount of time, storage and other resources required to execute the algorithm. The efficiency is measured with the help of **asymptotic notations**.
- The study of **change in performance** of the algorithm with the **change in the order of the input size** is defined as asymptotic analysis

The following 3 asymptotic notations are mostly used to represent the time complexity of algorithms.

- **Big-O notation**
- **Omega notation**
- **Theta notation**

Big-O Notation (O-notation)

- **Big-O notation** represents the upper bound of the running time of an algorithm. Thus, it gives the worst-case complexity of an algorithm.



The above expression can be described as a function **$f(n)$** belongs to the set **$O(g(n))$** if there exists a positive constant **c** such that it lies between **0** and **$cg(n)$** , for sufficiently large **n** .

For any value of **n** , the running time of an algorithm does not cross the time provided by **$O(g(n))$** .

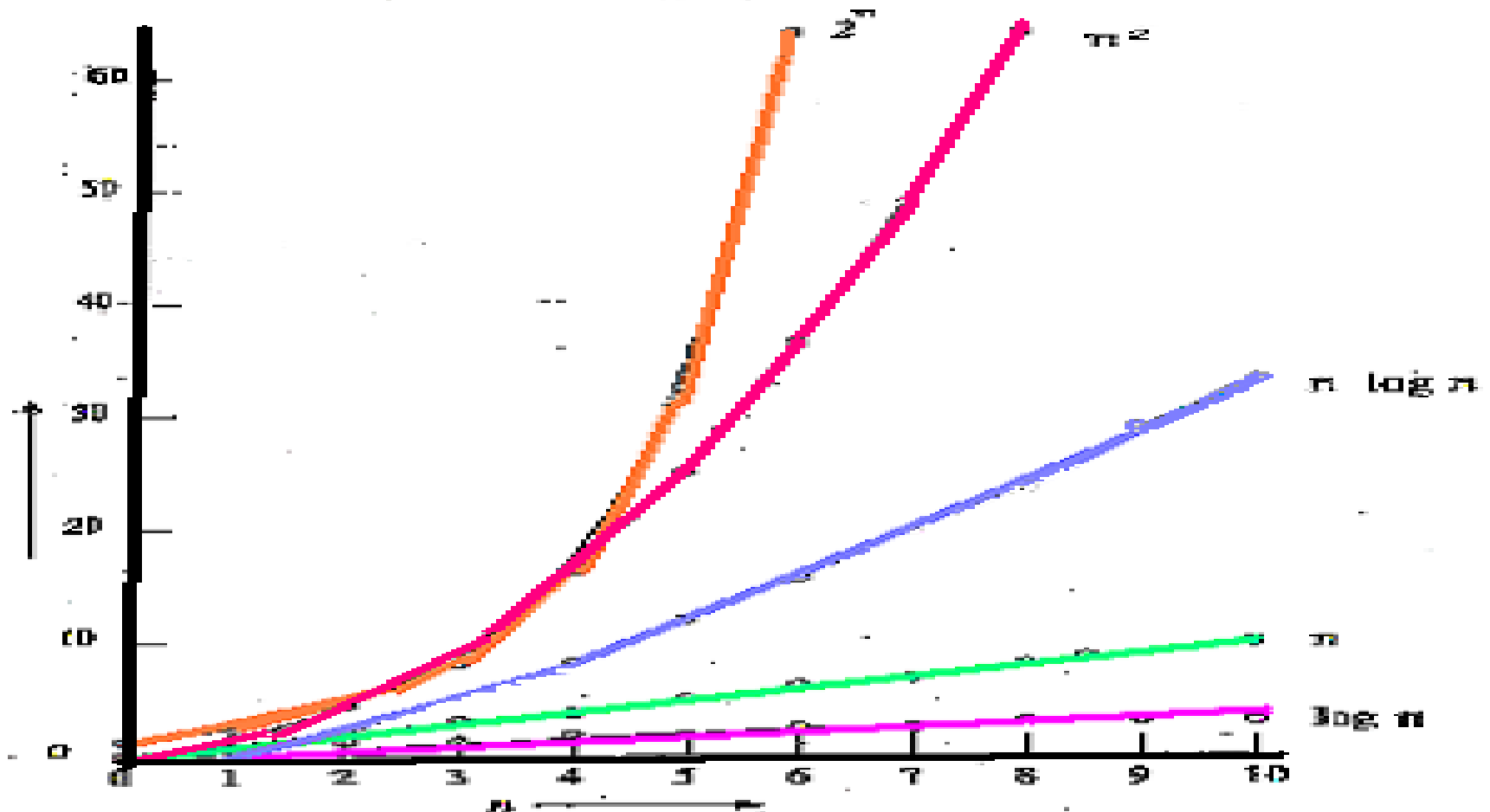
Since it gives the **worst-case running time** of an algorithm, it is widely used to analyze an algorithm as we are always interested in the worst-case scenario.

- Ignore all constant values
- Always measure $n \rightarrow$ infinity
- In case of upper bound, we need to take least upper bound
- In case of lower bound, we need to take greatest lower bound

BETTER**WORSE**

- ◆ $O(1)$ constant time
- ◆ $O(\log n)$ log time
- ◆ $O(n)$ linear time
- ◆ $O(n \log n)$ log linear time
- ◆ $O(n^2)$ quadratic time
- ◆ $O(n^2 \log n)$ log quadratic time
- ◆ $O(n^3)$ cubic time
- ◆ $O(2^n)$ exponential time
- ◆ $O(n^n)$ exponential time

Running Times- Comparison



Example 1

- $f(n)=3n+6$, prove that $f(n) \leq O C * g(n)$
- **You have find C and n_0 value such way that it should satisfy the condition.**
- $f(n)=3n+6$ and if we can take $g(n)=3n+6n$ then
- $3n+6 \leq 3n+6n$
- $3n+6 \leq 9n$, where $C=9$
- Whereas $n \geq n_0$, and here for all $n \geq n_0$, $n_0 \geq 1$
- **Can I take $g(n)$ as $3n+3$, if yes find C and n_0**

Example 2

- If $f(n)=2n+4$ then prove that $f(n)\leq C\cdot g(n)$
- **You have find C and n_0 value such way that it should satisfy the condition.**
- Lets take $g(n)= 2n+n$
- Where for all $n\geq n_0$
- $\rightarrow 2n+4\leq 2n+n$
- $\rightarrow 2n+4\leq 3n$ and $C=3$
- **The above holds true where $C=3$**
- **and $n_0\geq 4$**

n	f(n)	g(n)
1	6	3
2	8	6
3	10	9
4	12	12
5	14	15
6	16	18
7	18	21

Example 3

- If $f(n)=2n+4$ and let me take $g(n)= 2n+2n$
- $f(n)\leq C*g(n)$

- **You have find C and n_0 value such should satisfy the condition.**

- $\rightarrow 2n+4 \leq 4n$
- $\rightarrow 2n+4 \leq 4n$ and $C=4$
- Where for all $n \geq n_0$, and $n_0 \geq 2$

n	f(n)	g(n)
1	6	4
2	8	8
3	10	12
4	12	16
5	14	20
6	16	24
7	18	28

Example 4

```
public class Example{
    public static void main(String[] args) {
        int i,n=100;
        for(i=1; i<=n; i=i*2){
            System.out.println(i);
        }
    }
}
```

If n=1
1 time

If n=2
2 times

If n=4
3 times

If n=10
4

If n=20
5 times

'i' value → 1 2 4 8 16 32 64 128

2^0 2^1 2^2 2^3 2^4 2^5 2^6 2^7

Here k=0, 1, 2, 3, 4, 5, 6, 7

Prints → 1 1 1 1 1 1 1 1

$2^k \leq n$, apply both side log, then $k = \log_2 n$

$F(n) = O(\log_2 n)$

Example 5

- `for(i=1;i<=n;i=i*2)`
 - `print(“xxxx”);`

Value of “i” $\rightarrow 1, 2, 4, 8, 16, 32, 64, \dots$

$\rightarrow 2^0, 2^1, 2^2, 2^3, 2^4, 2^5, 2^6, \dots, 2^k$

$\rightarrow 2^k = n$

$\rightarrow$ Apply log both sides, $\log_2 (2^k) = \log_2 (n)$

$\rightarrow k \log_2 (2) = \log_2 (n)$

$\rightarrow k = \log_2 (n)$

$\rightarrow T(n) = \log_2 (n)$

Example 6

- for($i=1$; $i \leq n$; $i=i*5$)
 - print(“xxxx”);

Value of “ i ” $\rightarrow 1, 5, 25, 125, \dots$

$\rightarrow 5^0, 5^1, 5^2, 5^3, 5^4, 5^5, 5^6, \dots, 5^k$

$\rightarrow 5^k = n$

$\rightarrow$ Apply log both sides, $\log_5 (5^k) = \log_5 (n)$

$\rightarrow k \log_5 (5) = \log_5 (n)$

$\rightarrow k = \log_5 (n)$

$\rightarrow T(n) = \log_5 (n)$

Example 7

- for($i=n$; $i \geq 1$; $i=i/2$)
 - print(“xxxx”);

Value of “ i ” $\rightarrow n, n/2, n/4, n/8, n/16, n/32, \dots$

$\rightarrow n/2^0, n/2^1, n/2^2, n/2^3, n/2^4, \dots, n/2^k$

$\rightarrow n/2^k = 1$

$\rightarrow n = 2^k$

$\rightarrow$ Apply log both sides, $\log_2(2^k) = \log_2(n)$

$\rightarrow k \log_2(2) = \log_2(n)$

$\rightarrow k = \log_2(n)$

$\rightarrow T(n) = \log_2(n)$

Example 8

- for($i=2; i \leq n; i=i*i$)
 - print(“xxxx”);

Value of “i” $\rightarrow 2, 4, 16, 16*16, \dots$

$\rightarrow 2^{2^0}, 2^{2^1}, 2^{2^2}, 2^{2^3}, \dots, 2^{2^k}$

$\rightarrow 2^{2^k} = n$

$\rightarrow$ Apply log both sides, $\log_2(2^{2^k}) = \log_2(n)$

$\rightarrow 2^k \log_2(2) = \log_2(n)$

$\rightarrow 2^k = \log_2(n)$, again apply log

$\rightarrow \log_2(2^k) = \log_2(\log_2(n))$

$\rightarrow k \text{ **Log}_2(2)** = \log_2(\log_2(n)) \rightarrow k = \log_2(\log_2(n))$

Example 9

- for($i=n$; $i \geq 2$; $i=\text{sqrt}(i)$)
 - print(“xxxx”);

Value of “ i ” $\rightarrow n, n^{1/2}, n^{1/4}, n^{1/8}, \dots$
 $\rightarrow n^{1/(2^0)}, n^{1/(2^1)}, n^{1/(2^2)}, n^{1/(2^3)} \dots n^{1/(2^k)}$

$\rightarrow n^{1/2^k} \geq 2$

$\rightarrow$ Apply log both sides, $\log_2 (n^{1/2^k}) = \log_2 (2)$

$\rightarrow \frac{1}{2^k} \log_2 (n) = 1$

$\rightarrow \log_2 (n) = 2^k$

$\rightarrow$ Apply log both sides again

$\rightarrow \log_2 (2^k) = \log_2 (\log_2 (n))$

$\rightarrow k \log_2 (2) = \log_2 (\log_2 (n))$

$\rightarrow k = \log_2 (\log_2 (n))$

Example 10

```
for(i = n / 2; i <= n; i++)  
    for(j = 2; j <= n; j = j * 2)  
    {  
        k = k + n / 2;  
    }
```

For i value $\rightarrow n/2, (n/2)+1, (n/2)+2, (n/2)+3, \dots, n$

It iterates $n/2$ times which is same as n , when $n \rightarrow$ infinity

For j value $\rightarrow 2, 4, 8, 16, 32, 64, 128, \dots$

$2^1, 2^2, 2^3, 2^4, 2^5, 2^6, \dots, 2^k$

$\log_2(n)$

$T(n) = n \log_2(n)$

Example 10



- `for(i=1; i<=2n; i=i*2 )`
 - `print(“xxxx”);`

Example 9



- `for(i=1; i<=n2; i=i+10 )`
 - `print(“xxxx”);`

Example 10



- `for(i=n/2; i<=n; i=i*i )`
 - `print("xxxx");`

Example 11



- for(i=1; i<=n; i++)
 - for(i=1; i<=n; i++)
 - for(i=1; i<=n; i++)
 - Print(“XXX”);

Example 12



- `for(i=1; i<=n2; i=i*2)`
 - `for(j=n; j>=n/2; j=j/2)`
 - `for(k=2; k<=2n; k=k2)`
 - `Print("XXX");`

Example 13



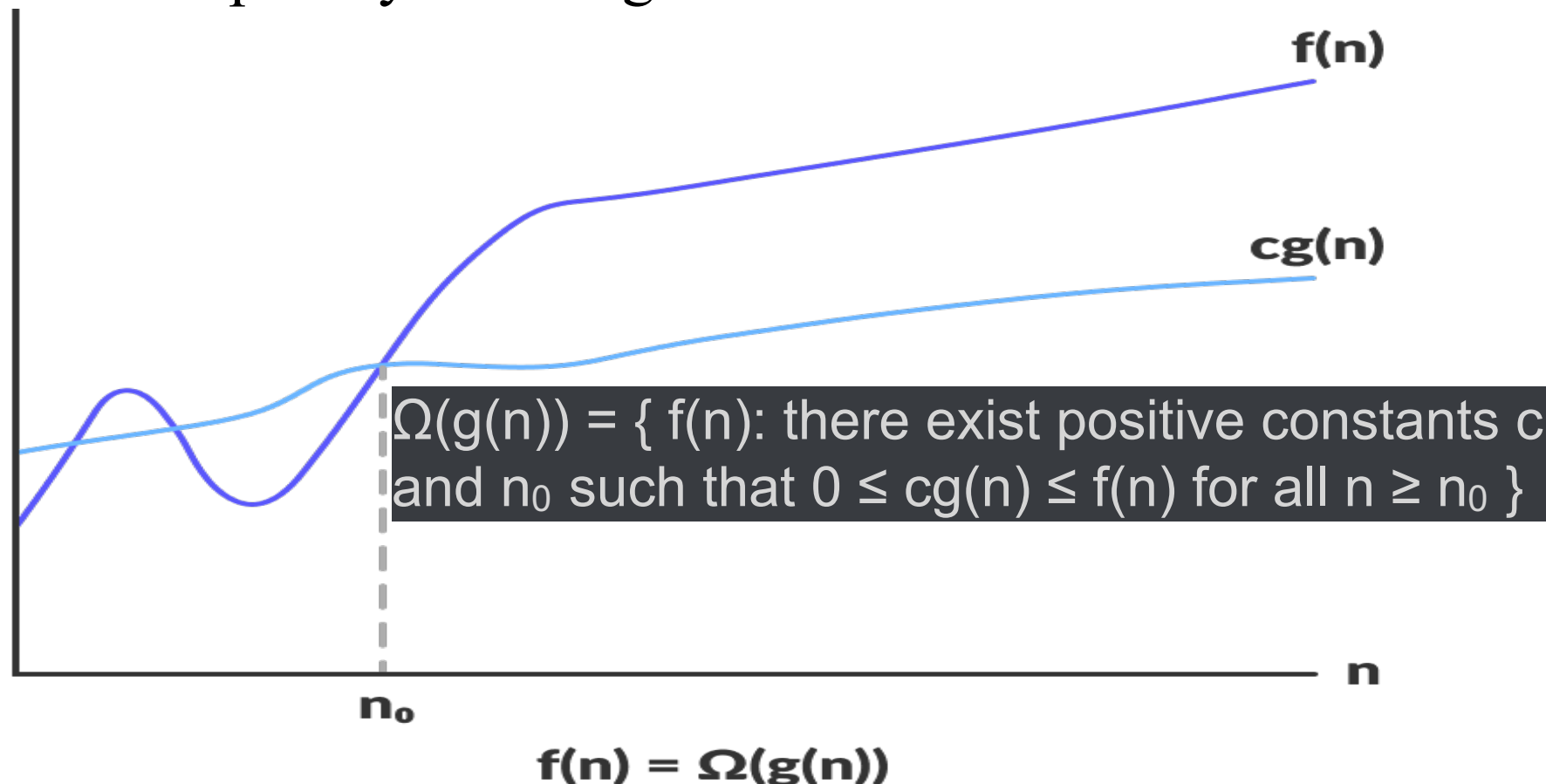
- for($i = n^2$; $i \geq 1$; $i = i/2$)
 - for($j = 1$; $j \leq n$; $j = j * 10$)
 - for($k = 1$; $k \leq 2^n$; $k = 2 * k$)
 - Print("XXX");

Prove the followings



- $3n+2=O(n)$
- $3n+2 \leq 4n$ for $n \geq 2$
- $3n+3=O(n)$
- $3n+3 \leq 4n$ for $n \geq 3$
- $100n+6=O(n)$
- $100n+6 \leq 101n$ for $n \geq 10$
- $10n^2+4n+2=O(n^2)$
- $10n^2+4n+2 \leq 11n^2$ for $n \geq 5$
- $6 \cdot 2^n + n^2 = O(2^n)$
- $6 \cdot 2^n + n^2 \leq 7 \cdot 2^n$ for $n \geq 4$

Omega notation represents the lower bound of the running time of an algorithm. Thus, it provides the best case complexity of an algorithm.



- The above expression can be described as a function $f(n)$ belongs to the set $\Omega(g(n))$ if there exists a positive constant c such that it lies above $cg(n)$, for sufficiently large n .
- For any value of n , the minimum time required by the algorithm is given by **Omega** $\Omega(g(n))$.
-

Example

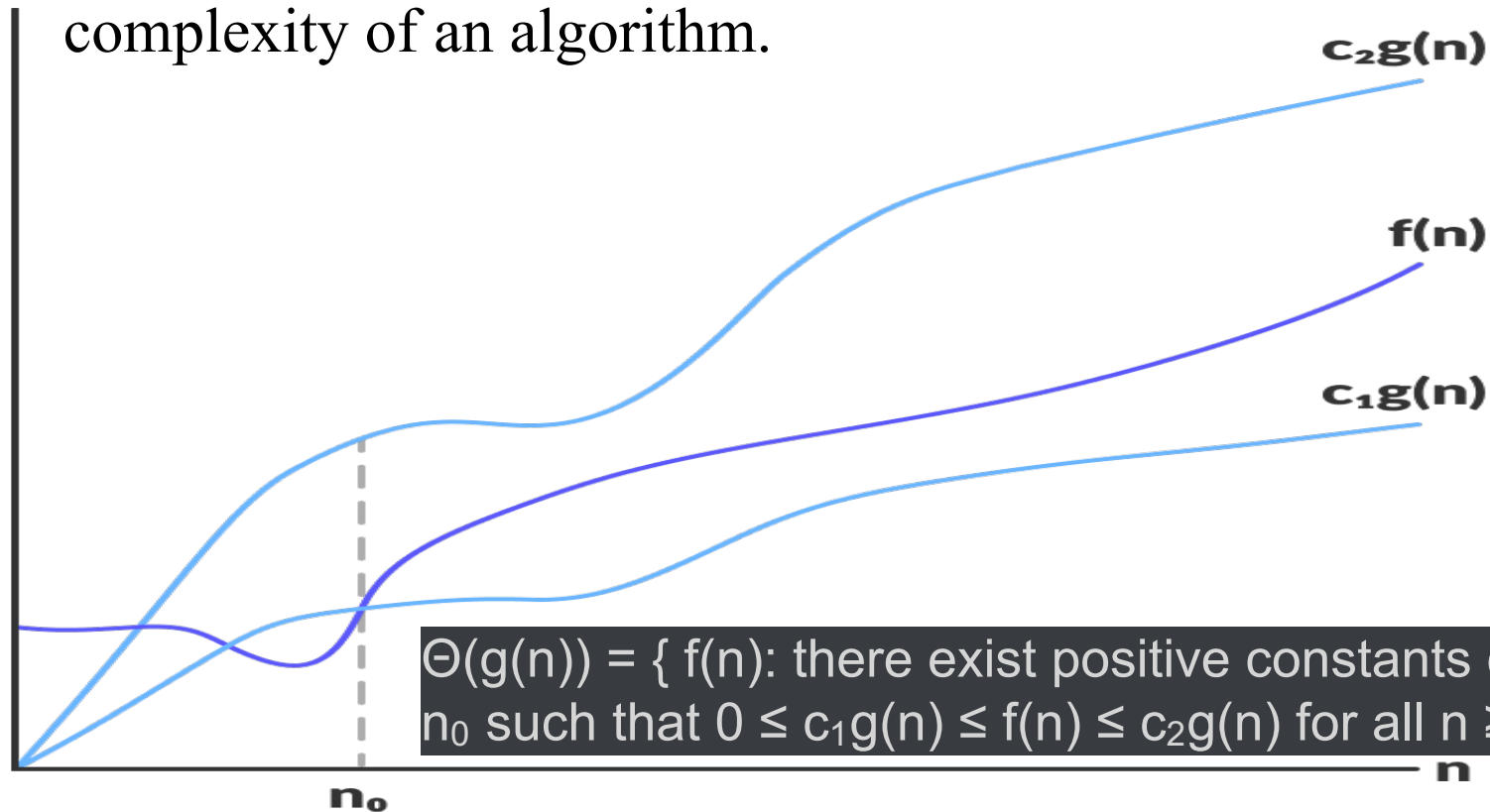


- Show that the following equalities are correct.
- $10n^2+4n+2 = \Omega(n^2)$
- $F(n) = 10n^2+4n+2$, Let $C*g(n) = 10n^2$
- $C*g(n) \leq f(n)$ or $f(n) \geq C*g(n)$
- $10n^2+4n+2 \geq 10n^2$
- $C=10$ and $n_0 \geq 1$

Theta Notation (Θ -notation)



Theta notation encloses the function from above and below. Since it represents the upper and the lower bound of the running time of an algorithm, it is used for analyzing the average-case complexity of an algorithm.



$$f(n) = \Theta(g(n))$$

- The above expression can be described as a function $f(n)$ belongs to the set $\Theta(g(n))$ if there exist positive constants c_1 and c_2 such that it can be sandwiched between $c_1g(n)$ and $c_2g(n)$, for sufficiently large n .
- If a function $f(n)$ lies anywhere in between $c_1g(n)$ and $c_2g(n)$ for all $n \geq n_0$, then $f(n)$ is said to be asymptotically tight bound.

Example

- Show that the following equalities are correct.
- $5n^2 - 6n = \Theta(n^2)$
- Let $F(n) = 5n^2 - 6n$, and we need to find $C1g(n)$ and $C2g(n)$ in such a way that: $C1 * g(n) \leq f(n) \leq C2 * g(n)$
- $C1 * g(n) = 5n^2 - n^2$ and $C2 * g(n) = 5n^2$
- $5n^2 - n^2 \leq 5n^2 - 6n \leq 5n^2$

N	C1g(n)	F(n)	C2g(n)
1	4	-1	5
2	16	8	20
3	36	27	45
4	64	40	80
5	100	50	125
6	144	144	180
7	196	203	245

Prove the followings

- $3n+2 = \Theta(n)$
- For $n \geq 2$, $c_1=3$ and $c_2=4$
- $10n^2 + 4n + 2 \geq \Theta(n^2)$
- For $n=?$, $c_1=10$ and $c_2=11$
- $10n^2 + 4n + 2 \geq 10n^2$
- $10n^2 + 4n + 2 \leq 11n^2$

The **space complexity** of an algorithm or a computer program is the amount of memory space required to solve an instance of the computational problem as a function of characteristics of the input. It is the memory required by an algorithm until it executes completely.

Similar to time complexity, space complexity is often expressed asymptotically in big O notation, such as $O(n)$, $O(n \log n)$, $O(2^n)$ etc., where n is a characteristic of the input influencing space complexity.

Example



```
void sumFun( )  
{   int a, b, c, sum;  
    sum = a + b + c;  
    return(sum);  
}
```

$S(n)=4 \rightarrow S(n)=C$

```
void sumFun(int a, int b)  
{   int sum;  
    sum = a + b;  
    return(sum);  
}
```

$S(n)=1 \rightarrow S(n)=C$

Example

```
int sum(int arr[n], int n) {  
    int B[n], k, i = 0;  
    for(k = 0; k < n; k++)  
    {  
        i = i + arr[k];  
        B[i]=arr[i]  
    }  
    return(i);  
}
```



$n+1+1$ memory spaces

=====

Total= $O(n)$

```
int factorial(int n) {  
    if (n == 0)  
        return 1;  
    return n * factorial(n - 1);  
}
```

$F(5) \rightarrow F(4)$

$F(4) \rightarrow F(3)$

$F(3) \rightarrow F(2)$

$F(2) \rightarrow F(1)$

$F(1) \rightarrow F(0)$

Space Complexity = $O(n)$

In iteration, there is a repeated execution of the set of instructions. In Iteration, loops are used to execute the set of instructions repetitively until the condition is false.

Recursion is the process of calling a function itself within its own code.

- Recurrence is an equation or inequality that describes a function in terms of its value on smaller inputs, and one or more base cases
 - Useful for analyzing recurrent algorithms
 - Make it easier to compare the complexity of algorithms
 - Methods for solving recurrences
 - **Substitution method**
 - **Recursion tree method**
 - **Master method**
 - **Iteration method**

3.1 Methods for Resolving Recurrence Relations

Recurrence relations can be resolved with any of the following four methods:

1. Substitution Method (Guess-and-Verify)
2. Iteration Method.
3. Recursion Tree Method.
4. Master Method.

- Use mathematical induction to derive an answer
- Derive a function of n (or other variables used to express the size of the problem) that is not a recurrence so we can establish an upper and/or lower bound on the recurrence. May get an exact solution or may just get upper or lower bounds on the solution

Steps

- **Guess the form of the solution**
- **Use the mathematical induction to find the boundary condition and shows that the guess is correct.**

Recurrence Relation

	s/e	freq	= total
int factorial(int n) {			
if (n == 1) —————→	1	1	= 1
return 1; —————→	1	1	= 1
return n * factorial(n - 1); —————→	1	1+f(n-1)	= T(n-1)+1
}			

$$\begin{aligned}
 T(n) &= \begin{cases} \text{if } n == 1, & 1 \\ \text{else} & T(n-1) + 3 \end{cases} \\
 &= 1 + T(n-1)
 \end{aligned}$$

Many algorithms are recursive. When we analyze them, we get a recurrence relation for time complexity.

Recurrence Relation

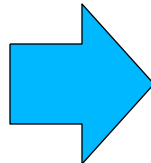
	s/e	freq	= total
int Test(int n) {			
if (n == 1) →	1	1	= 1
S.o.p(n); →	1	1	= 1
else			
{ for(i=0; i<n; i++) →	1	n+1	=n+1
S.o.p(n); →	1	n	=n
Test(n - 1); →	1	T(n-1)	=1+T(n-1)
}			-----
}			

$T(n) = \begin{cases} 1 & \text{if } n == 1 \\ 2n+1 & \text{else} \end{cases}$

$+T(n-1)$

```
void test(int n)
{
    if(n>1)
        sop(n)
        test(n/2);
    else
        s.o.p(n);
}
```

$$\begin{aligned}T(n) &= T(n/2) + 1 \\T(n) &= T(n/4) + 1 + 1 \\&= T(n/8) + 3 \\&= T(n/16) + 4 \\&= T(n/32) + 5\end{aligned}$$



$$\begin{aligned}T(n) &= T(n/2) + 1 \\T(n) &= T(n/2^2) + 2 \\&= T(n/2^3) + 3 \\&= T(n/2^4) + 4 \\&= T(n/2^5) + 5\end{aligned}$$

Let $2^k = n$

$$\begin{aligned}&= T(n/2^k) + k \\&= T(1) + k\end{aligned}$$

If $2^k = n$, then $k = ?$

Apply log both side

$$k = \log_2 n$$
$$T(n) = \log(n)$$

Example1

```
A(n)
{
if (n>1)
return (A(n-1))
}
```

$$T(n) = 1 + T(n-1) \quad \dots \quad \text{Eqn. (1)}$$

Step1: Substitute $n-1$ at the place of n in Eqn. (1)

$$T(n-1) = 1 + T(n-2) \quad \dots \quad \text{Eqn. (2)}$$

Step2: Substitute $n-2$ at the place of n in Eqn. (1)

$$T(n-2) = 1 + T(n-3) \quad \dots \quad \text{Eqn. (3)}$$

$$T(n) = 1 + 1 + T(n-2) = 2 + T(n-2) \quad \dots \quad \text{Eqn. (4)}$$

Recursion Example 2

$T(n)=T(n-1)+3$, for all value of $n>0$

$T(0)=1$

$T(n)=T(n-1)+3$

$T(n-1)=T(n-2)+3$

$\rightarrow T(n)=T(n-2)+3+3 \Rightarrow T(n-2)+6$

$T(n-2)=T(n-3)+3$

$\rightarrow T(n)=T(n-3)+3+6 \Rightarrow T(n-3)+9$

$\rightarrow T(n)=T(n-3)+3*3$

$\rightarrow T(n)=T(n-4)+4*3$

$\rightarrow T(n)=T(n-5)+5*3$

$\rightarrow T(n)=T(n-k)+3*k$

Let $n=k$, then $n-k=0$

$T(n)=T(n-k)+3k$

$T(n)=T(0)+3k$

If $n-k=0$, then $n=k$

$=T(0)+3n$

$T(0)=1$

$T(n)=1+3n$, because $k=n$

$T(n)=O(n)$

Recursion Example 3



$$T(n)=n+T(n-1)$$

$$=n+(n-1)+T(n-2)$$

$$=n+(n-1)+(n-2)+T(n-3)$$

$$= n+(n-1)+(n-2)+(n-3)+T(n-4)$$

$$= n+(n-1)+(n-2)+(n-3)+(n-4)+T(n-5)$$

$$=n+(n-1)+(n-2)+ \dots +(n-3)+(n-4)+(n-5)+ \dots ..(n-(k-1))+T(n-k)$$

$$\dots\dots\dots\text{let } k=n$$

$$= n+(n-1)+(n-2)+ \dots +(n-3)+(n-4)+(n-5)+ \dots \dots +(1)+T(0)$$

$$=\{n*(n+1)\}/2$$

$$=O(n^2)$$

Recursion Example 5



$$T(n)=2T(n/2)+n \text{ and } T(1)=1$$

Replace n with $n/2$,

$$T(n/2)=2T(n/4)+n/2,$$

$$T(n)= 2[2T(n/4)+n/2]+n$$

$$\rightarrow 4T(n/4)+n+n$$

$$2^2T(n/2^2)+2n$$

Replace n with $n/4$,

$$T(n/4)=2T(n/8)+n/4,$$

$$T(n)= 4[2T(n/8)+n/4]+2n$$

$$\rightarrow 8T(n/8)+n+2n$$

$$\text{we can write } \rightarrow 2^3T(n/2^3)+3n$$

we can write $\rightarrow$

Recursion Example 5



we can write $\rightarrow 2^3 T(n/2^3) + 3n$

So, we guess the General equation,

$$T(n) = 2^k * T(n/2^k) + kn$$

Let $n = 2^k$

$$T(n) = 2^k T(1) + kn$$

then, $n = 2^k$ Apply log both sides, $k = \log_2(n)$

Now $T(n) = n + n \log_2(n)$

$$\rightarrow n + n \log_2(n) \rightarrow O(n \log_2(n))$$

- Main disadvantage of Substitution method is that it is always difficult to come up with a good guess.
- Recursion tree method allows you make a good guess for the substitution method.
- Allows to visualize the process of iterating the recurrence

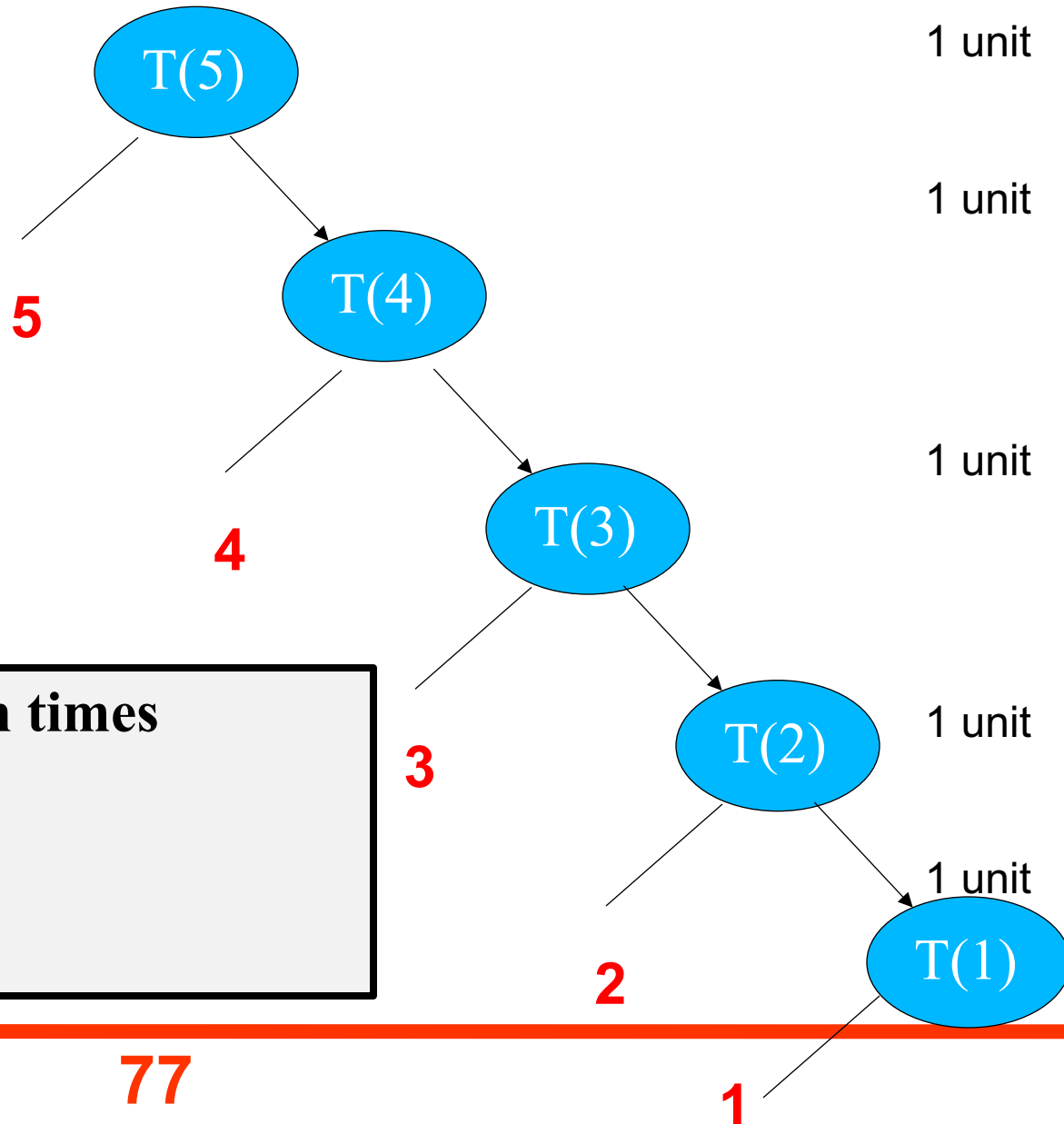
Steps

- **Convert the recurrence into a tree.**
- Each node represents the cost of a single sub problem somewhere in the set of recursive function invocations
- Sum the costs within each level of the tree to obtain a set of per-level costs
- Sum all the per-level costs to determine the total cost of all levels of the recursion.

Recursive Example 1

```
void Test(int n)
{
    if (n == 1)
        S.O.P(1);
    else
    { S.O.P(n);
      Test(n - 1);
    }
}
```

$T(n) = 1 + 1 + 1 + 1 + \dots n \text{ times}$
 $= n \text{ Times}$
 $= O(n)$



Recursive: Using Recurrence Graph



```
void Test(int n) {
```

```
    if (n == 1)
```

```
        S.O.P(1);
```

```
    else
```

```
    { for(int i=1;i<=n; i++)
```

```
        S.O.P(n);
```

```
        Test(n - 1);
```

```
    }
```

```
}  $T(n) = n + (n-1) + (n-2) + (n-3) + \dots + 1$ 
```

```
 $T(n) = (n * (n+1)) / 2$ 
```

```
 $= (n^2 + n) / 2$ 
```

```
 $= O(n^2)$ 
```

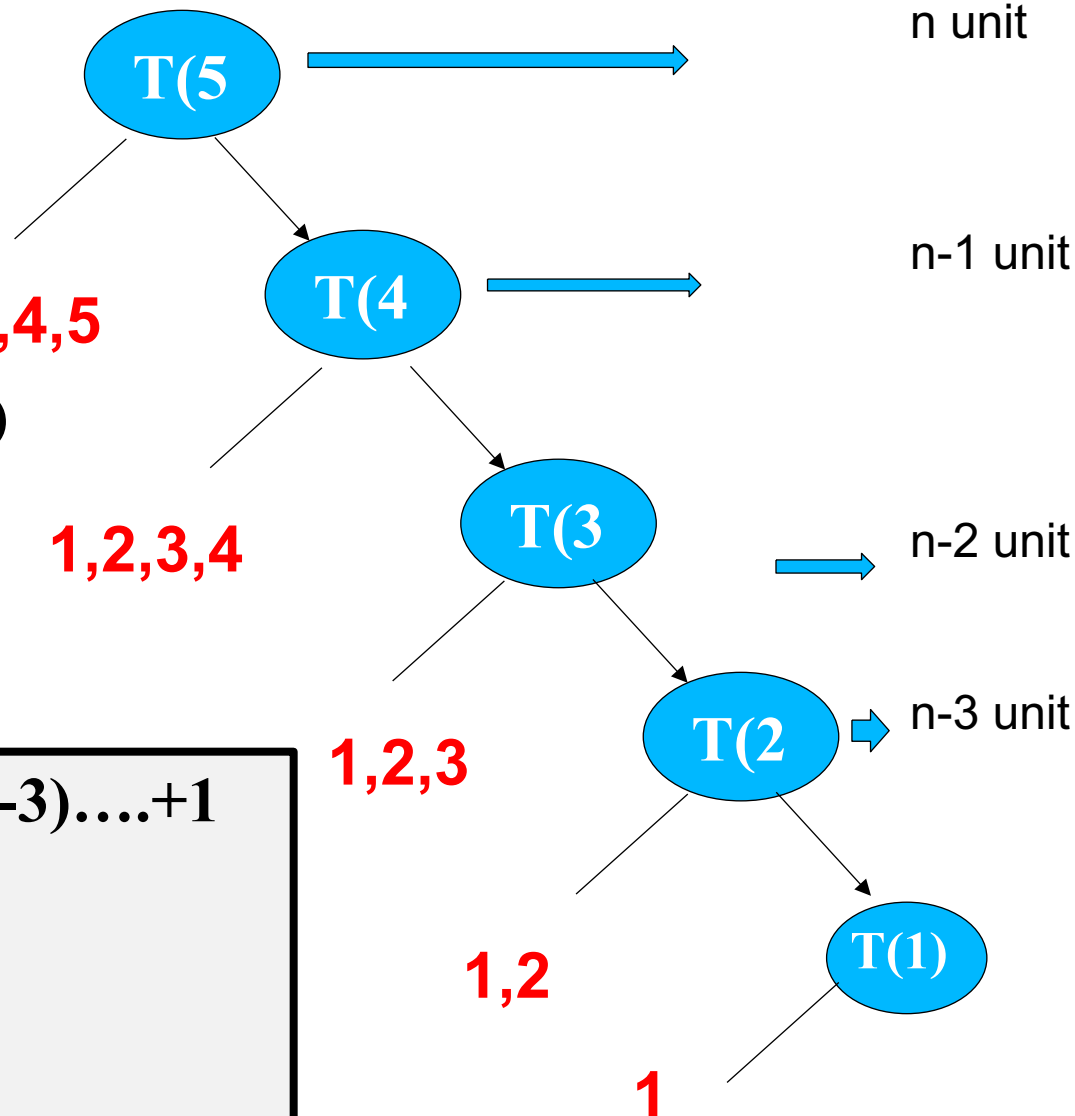
1,2,3,4,5

1,2,3,4

1,2,3

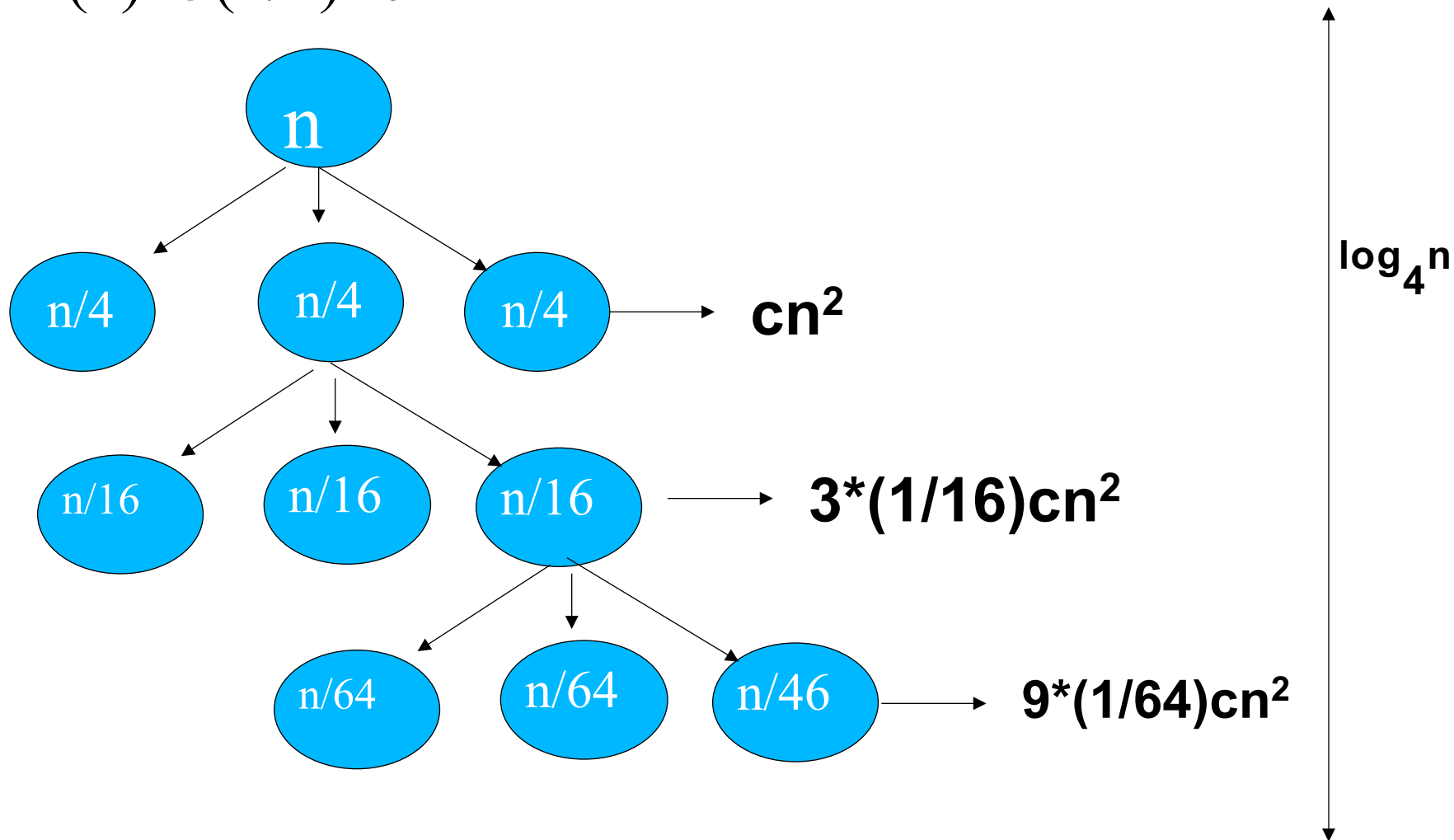
1,2

1



Example

$$T(n) = 3(n/4) + cn^2$$



$$T(n) = cn^2 + (3/16)^2 cn^2 + (3/16)^3 cn^2 + (3/16)^4 cn^2 + \dots$$

$$a = cn^2, r = (3/16), m = \log n$$

The sum of first n terms of the Geometric progression is

$$S_n = a + ar + ar^2 + ar^3 + \dots + ar^{n-2} + ar^{n-1} \text{ -----} > (1)$$

The sum of the first m terms of a geometric progression is:

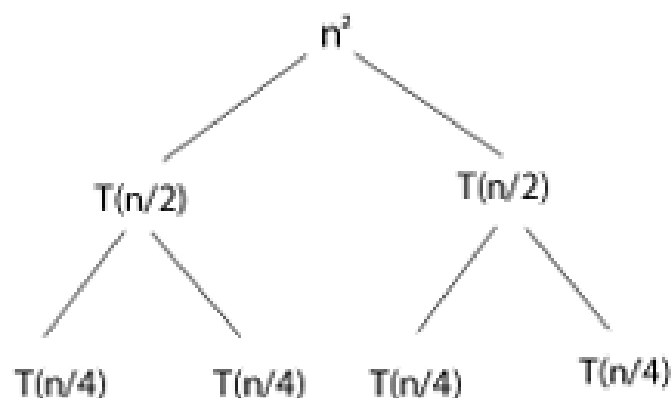
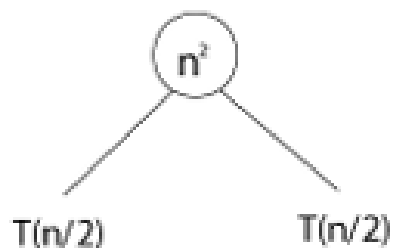
$$S_m = a \frac{r^m - 1}{r - 1}.$$

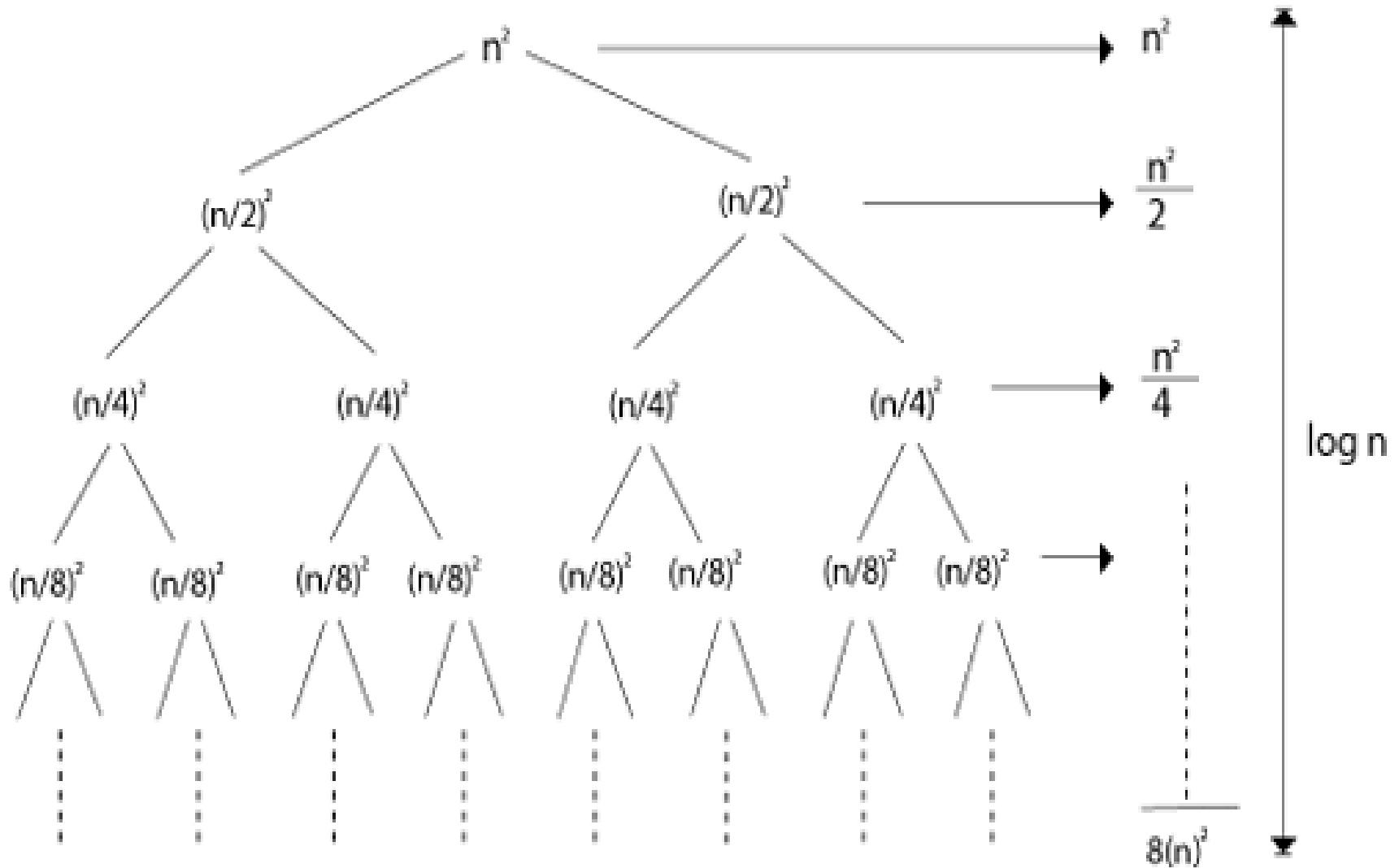
$$T(n) = Cn^2 \cdot \frac{\left(\frac{3}{16}\right)^{\log n} - 1}{\frac{3}{16} - 1}.$$

Consider $T(n) = 2T\left(\frac{n}{2}\right) + n^2$

We have to obtain the asymptotic bound using recursion tree method.

Solution: The Recursion tree for the above recurrence is





$T(n) = n^2 + (n^2/2) + (n^2/4) + (n^2/8) + \dots$ Log n times

$T(n) = n^2(1 + (1/2) + (1/4) + (1/8) + \dots)$ Log n times

$T(n) = n^2(1/2^0 + (1/2^1) + (1/2^2) + (1/2^3) + \dots)$ Log n times

$$\leq n^2 \sum_{i=0}^{\infty} \left(\frac{1}{2^i}\right)$$

$$\leq n^2 \left(\frac{1}{1 - \frac{1}{2}}\right) \leq 2n^2$$

$$T(n) = \Theta(n^2)$$

- The number of terms is $\log n$ (base 2, assumed unless otherwise stated).

The sum of the first m terms of a geometric progression is:

$$S_m = a \frac{r^m - 1}{r - 1}.$$

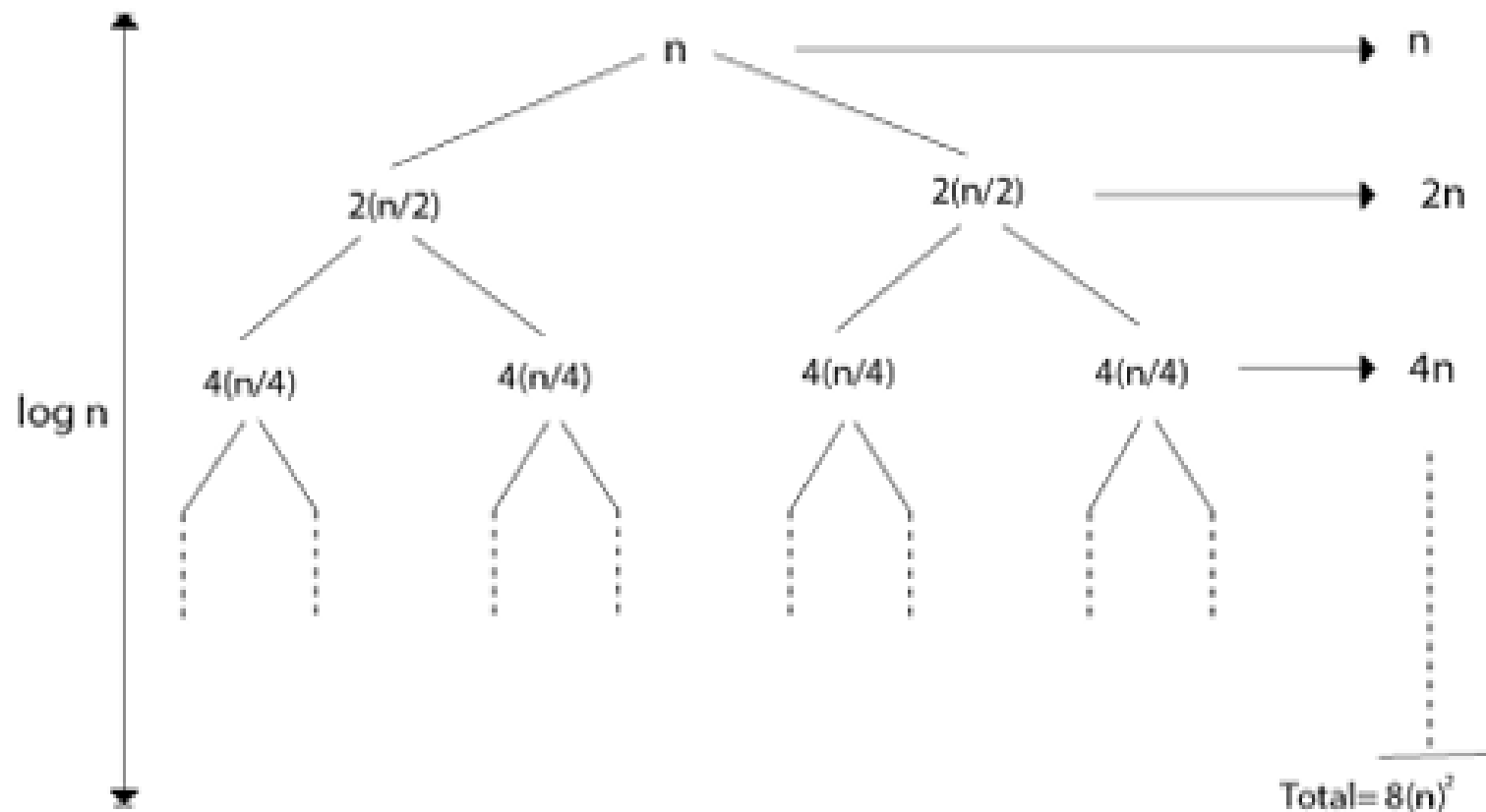
Example

Example 2: Consider the following recurrence

$$T(n) = 4T\left(\frac{n}{2}\right) + n$$

Obtain the asymptotic bound using recursion tree method.

Solution: The recursion trees for the above recurrence



$$A^{\log_c b} = b^{\log_c A}$$

$$2^{\log_2 n} = n^{\log_2 2}$$

We have $n + 2n + 4n + \dots \log_2 n$ times

$$= n (1 + 2 + 4 + \dots \log_2 n \text{ times})$$

$$= n \frac{(2^{\log_2 n} - 1)}{(2 - 1)} = \frac{n(n - 1)}{1} = n^2 - n = \theta(n^2)$$

$$T(n) = \theta(n^2)$$

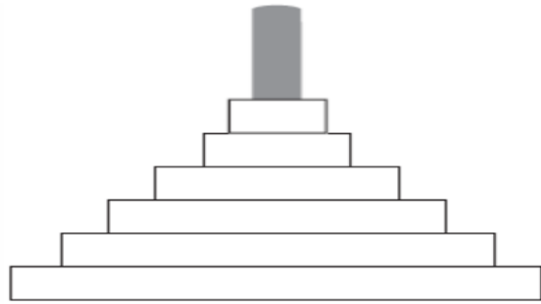
- The first term $a = 1$,
- The common ratio $r = 2$,
- The number of terms is $\log n$ (base 2, assumed unless otherwise stated).

The sum of the first m terms of a geometric progression is:

$$S_m = a \frac{r^m - 1}{r - 1}.$$

For this series, $a = 1$, $r = 2$, and $m = \log n$. Substituting these values:

- In this puzzle, we have n disks of different sizes that can slide onto any of three pegs.
- Initially, all the disks are on the first peg in order of size, the largest on the bottom and the smallest on top.
- The goal is to move all the disks to the third peg, using the second one as an auxiliary, if necessary.
- We can move only one disk at a time, and it is forbidden to place a larger disk on top of a smaller one



B



A



E

```
void ToH(int n, char B, char E, char A){  
    if (n == 0) { return; }  
    ToH(n-1, B, E, A);  
    S.O.P("Move disk " + n + "From" + B + "To" + E);  
    ToH(n-1, A, B, E);  
}
```

$T(n) = T(n-1) + T(n-1) + c$ for $n > 1$ and $T(1) = 1$
 $T(n) = 2T(n-1) + c$

Time Complexity

$$T(n) = 2T(n-1) + 1$$

$T(n-1) \rightarrow 2T(n-2) + 1$, then substitute

$$\begin{aligned} T(n) &= 2[2T(n-2) + 1] + 1 \\ &= 4T(n-2) + 2 + 1 \\ &= 2^2T(n-2) + 2 + 1 \end{aligned}$$

$T(n-2) \rightarrow 2T(n-3) + 1$, then substitute

$$\begin{aligned} T(n) &= 2^2[2T(n-3) + 1] + 2 + 1 \\ &= 2^3T(n-3) + 2^2 + 2 + 1 \end{aligned}$$

after i substitutions, we get

$$T(n) = 2^iT(n-i) + 2^{i-1} + 2^{i-2} + 2^{i-3} + 2^{i-4} + \dots + 2 + 1$$

- $T(n) = 2^i T(n-i) + 2^{i-1} + 2^{i-2} + 2^{i-3} + 2^{i-4} + \dots + 2 + 1$
- $= 2^i T(n-i) + 2^i - 1$
- after $k-1$, where $k=n$ substitutions, we get
- $T(n) = 2^{k-1} T(n-(k-1)) + 2^{k-1} - 1$
- $T(n) = 2^{n-1} T(n-(n-1)) + 2^{n-1} - 1$
- $T(n) = 2^{n-1} T(1) + 2^{n-1} - 1$
- $T(n) = 2^n - 1$

Example: $2^3 + 2^2 + 2^1 + 2^0 = 2^4 - 1$

- The **Master Method** is a commonly used approach for analyzing the time complexity of divide-and-conquer algorithms.
- It applies to recurrence relations of the form:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Where:

$a \geq 1$: The number of subproblems.

$b > 1$: The factor by which the problem size is divided in each recursive step.

$f(n)$: The cost of the work done outside the recursive calls (e.g., merging, dividing).

- Master's theorem can only be applied on decreasing and dividing recurring functions. If the relation is not decreasing or dividing, master's theorem must not be applied.

Master's Theorem for Dividing Functions

- $T(n) = aT(n/b) + f(n)$

where, $a \geq 1$ and $b \geq 1$,

n – size of the problem

a – number of sub-problems in the recursion

n/b – size of the sub problems based on the assumption that all sub-problems are of the same size.

$f(n)$ – represents the cost of work done outside the recursion

If the recurrence relation is in the above given form, then there are three cases in the master theorem to determine the asymptotic notations –

- $T(n) = aT(n/b) + f(n)$
- The Master Theorem provides three cases to determine the asymptotic bounds of $(T(n))$:

CASE 1: if $f(n) = O(n^{\log_b a - E})$ for some constant where $E > 0$, then

$$T(n) = \Theta(n^{\log_b a})$$

If $f(n)$ grows slower than $n^{\log_b a}$, the total time complexity is determined by the recursive part:

$$T(n) = \Theta(n^{\log_b a})$$

- $T(n) = aT(n/b) + f(n)$
- **CASE 2:** if $f(n) = \Theta(n^{\log_b a})$ then
 - $T(n) = \Theta(n^{\log_b a} * \log_b n)$

If $f(n)$ grows at the same rate as $n^{\log_b a}$, the total time complexity includes the contribution from $f(n)$:

$$T(n) = \Theta(n^{\log_b a} \log n)$$

- $T(n) = aT(n/b) + f(n)$

CASE 3: if $f(n) = \Omega(n^{(\log_b a) + E})$ then for some constant $E > 0$ and if $f(n/b) \leq C f(n)$ for some constant $C < 1$ and all sufficiently large n , then

- $T(n) = \Theta(f(n))$

If $f(n)$ grows faster than $n^{\log_b a}$

- $T(n) = aT(n/b) + f(n)$
- Where, we interpret n/b as n/b or n/b then $T(n)$ can be
- **CASE 1:** if $f(n) = O(n^{(\log_b a) - E})$ for some constant where $E > 0$, then $T(n) = \Theta(n^{\log_b a})$
- **CASE 2:** if $f(n) = \Theta(n^{\log_b a})$ then
 - $T(n) = \Theta(n^{\log_b a} * \log_b n)$
- **CASE 3:** if $f(n) = \Omega(n^{(\log_b a) + E})$ then for some constant and if $f(n/b) \leq Cf(n)$ for some constant $C < 1$ and all sufficiently large n , then
 - $T(n) = \Theta(f(n))$

Example 1

- $T(n) = 4T(n/2) + n$
- As given $T(n) = aT(n/b) + f(n)$
- Here we get $a=4$, $b=2$ and $f(n)=n$
- Check for CASE1
- $f(n) = O(n^{\log_b a - \epsilon})$
- $\rightarrow O(n^{\log_2 4 - \epsilon})$
- $\rightarrow O(n^{\log_2 2 - \epsilon})$
- $\rightarrow O(n^{2 - \epsilon})$
- $\rightarrow O(n^2)$
- $f(n) = n$, so
- $n = O(n^{2 - \epsilon})$, Here $\epsilon > 0$, so it satisfies case 1
- $T(n) = \Theta(n^{\log_b a})$
- **$T(n) = \Theta(n^2)$**

Example 2

- $T(n) = 4T(n/2) + n^2$
- As given $T(n) = aT(n/b) + f(n)$
- Here we get $a=4$, $b=2$ and $f(n)=n^2$
- Check for CASE 1
- $f(n) = O(n^{\log_b a - \epsilon})$
- $\rightarrow O(n^{\log_2 4 - \epsilon})$
- $\rightarrow O(n^{\log_2 2 - \epsilon})$
- $\rightarrow O(n^{2 \log_2 2 - \epsilon})$
- $\rightarrow O(n^2)$
- **$n^2 = O(n^{2-\epsilon})$, so it is not satisfy case 1**
- Check for CASE 2:
- **$n^2 = O(n^2)$, so it is satisfy case 2**
- $T(n) = \Theta(n^{\log_b a} * \log n)$
- **$T(n) = \Theta(n^2 * \log n)$**

Example 3

- $T(n) = 7T(n/2) + n^2$
- As given $T(n) = aT(n/b) + f(n)$
- Here we get $a=7$, $b=2$ and $f(n)=n^2$
- Check for CASE1
 - $f(n) = O(n^{\log_b a - \epsilon})$
 - $\rightarrow O(n^{\log_2 7}) \rightarrow$ Here $\log_2 7 = 2.81$
 - $\rightarrow O(n^{2.81})$
 - **$n^2 = O(n^{2.81 - \epsilon})$, so it is satisfy case 1**
- $T(n) = \Theta(n^{\log_b a})$
- **$T(n) = \Theta(n^{2.81})$**

Example 4

- $T(n) = 8T(n/2) + n^2$
- As given $T(n) = aT(n/b) + f(n)$
- Here we get $a=8$, $b=2$ and $f(n)=n^2$
- Check for CASE1
 - $f(n) = O(n^{\log_b a - \epsilon})$
 - $\rightarrow O(n^{\log_2 8})$
 - $\rightarrow O(n^{\log_2 23})$
 - $\rightarrow O(n^{3\log_2 2})$
 - $\rightarrow O(n^3)$
 - **$n^2 = O(n^{3-\epsilon})$, so it is satisfy case 1**
- $T(n) = \Theta(n^{\log_b a})$
- **$T(n) = \Theta(n^3)$**

Example 5

- $T(n) = 4T(n/2) + n^3$
- As given $T(n) = aT(n/b) + f(n)$
- Here we get $a=4$, $b=2$ and $f(n)=n^3$
- **Check for CASE 1**
- $f(n) = O(n^{\log_b a - \epsilon})$
- $\rightarrow O(n^{\log_2 4})$
- $\rightarrow O(n^{\log_2 2})$
- $\rightarrow O(n^{2 \log_2 2})$
- $\rightarrow O(n^2)$
- **$n^3 = O(n^{2-\epsilon})$, so it is not satisfy case 1**

- **Check for CASE 2**
 - $f(n) = O(n^{\log_b a})$
 - $\rightarrow O(n^{\log_2 4})$
 - $\rightarrow O(n^{\log_2 22})$
 - $\rightarrow O(n^{2 \log_2 2})$
 - $\rightarrow O(n^2)$
 - $n^3 = O(n^2)$, so it is not satisfy case 2
- **Check for CASE 3**
 - $n^3 = O(n^{2+E})$, so it is satisfy case 3, $E > 0$, then
 - $T(n) = \Theta(f(n))$
 - $T(n) = \Theta(n^3)$

$$T(n) = aT(n-b) + f(n)$$

where, $a \geq 1$ and $b > 1$, $f(n)$ is asymptotically positive

n – size of the problem

a – number of sub-problems in the recursion

$n-b$ – size of the sub problems based on the assumption that all sub-problems are of the same size.

$f(n)$ – represents the cost of work done outside the recursion $\rightarrow \Theta(n^k)$, where $k \geq 0$.

If the recurrence relation is in the above given form, then there are three cases in the master theorem to determine the asymptotic notations

if $a = 1$, $T(n) = O(n^{k+1})$

if $a > 1$, $T(n) = O(a^{n/b} * n^k)$

if $a < 1$, $T(n) = O(n^k)$

Example 1

$$T(n) = T(n-1) + n^2$$

Where $a = 1$, $b = 1$ and $f(n) = O(n^k) = n^2$,

Given $k = 2$.

Since $a = 1$, case 1 must be applied for this equation.

if $a = 1$, $T(n) = O(n^{k+1})$

$$T(n) = O(n^{k+1}) = n^{2+1} = n^3$$

$$T(n) = O(n^3).$$

Example 2

$$T(n) = 2T(n-1) + n$$

Where $a = 2$, $b = 1$ and $f(n) = O(n^k) = n^1$,

Given $k = 1$.

Since $a > 1$, case 2 must be applied for this equation.

if $a > 1$, $T(n) = O(a^{n/b} * n^k)$

$$T(n) = O(a^{n/b} * n^k) \rightarrow 2^{n/1} * n \rightarrow n * 2^n$$

$$T(n) = O(n * 2^n).$$

$$T(n) = aT(n/b) + f(n)$$

where, $a \geq 1$ and $b > 1$,

n – size of the problem

a – number of sub-problems in the recursion

n/b – size of the sub problems based on the assumption that all sub-problems are of the same size.

$f(n)$ – represents the cost of work done outside the recursion $\rightarrow \Theta(n^k \log n^p)$, where $k \geq 0$ and p is a real number;

If the recurrence relation is in the above given form, then there are three cases in the master theorem to determine the asymptotic notations

End